Curso Pandas

Lección 1: Introducción a Pandas

```python
import pandas as pd

# Crear una Series simple (unidimensional)
serie_minutos = pd.Series([12, 45, 78])
# La Series tiene un índice por defecto: 0, 1, 2

# Series con índices personalizados
serie_jugadores = pd.Series(
    ["Messi", "Mbappé", "Haaland"],
    index=["ARG", "FRA", "NOR"]
)

# Acceder a un valor usando el índice personalizado
serie_jugadores["FRA"]

# Crear un DataFrame desde un diccionario
data = {
    "id": [1, 2, 3],
    "minute": [12, 45, 78],
    "player": ["Messi", "Mbappé", "Haaland"],
    "xG": [0.79, 0.12, 0.67],
    "result": ["Goal", "Miss", "Goal"]
}
```

```python
df = pd.DataFrame(data)

# Agregar una nueva columna al DataFrame
df["shotType"] = ["LeftFoot", "RightFoot", "Header"]
df["h_a"] = ["h", "a", "h"]
df["year"] = [2021, 2022, 2022]

# Crear un DataFrame a partir de una lista de diccionarios
otros_datos = [
    {"id": 4, "minute": 30, "player": "Salah", "xG": 0.34, "result": "Saved"},
    {"id": 5, "minute": 61, "player": "Lewandowski", "xG": 0.52, "result": "Goal"}
]
df2 = pd.DataFrame(otros_datos)

# Unir ambos DataFrames
df_total = pd.concat([df, df2], ignore_index=True)
```

Ejercicios propuestos

- Crear una Series con los valores de xG [0.65, 0.21, 0.48] y nombres de jugadores como índice.
- Crear un DataFrame con al menos 5 filas y columnas id, minute, player, xG, result.
- Agregar columnas h_a, year y shotType al DataFrame anterior.
- Concatenar dos DataFrames y mostrar las primeras 3 filas.

Lección 2: Lectura y escritura de archivos

```python
# Guardar DataFrame en un archivo CSV
df_total.to_csv("datos_disparos.csv", index=False)

# Leer un archivo CSV
df_leido = pd.read_csv("datos_disparos.csv")

# Guardar como Excel
df_total.to_excel("datos_disparos.xlsx", index=False)

# Leer un archivo Excel
df_excel = pd.read_excel("datos_disparos.xlsx")
```

Ejercicios propuestos

- Guardar el DataFrame como mis_disparos.csv y leerlo nuevamente.
- Crear y leer un archivo Excel con los mismos datos.
- Mostrar solo las 2 primeras filas del archivo leído.

Lección 3: Exploración de datos

```
# Ver primeras y últimas filas
df_total.head()       # Primeras 5
df_total.tail(2)      # Últimas 2

# Información general: tipos, nulos
df_total.info()

# Estadísticas básicas de columnas numéricas
df_total.describe()

# Ver nombres de columnas
df_total.columns

# Cambiar nombres de columna a minúsculas
df_total.columns = df_total.columns.str.lower()
```

Ejercicios – Lección 3

- Usar info() y explicar qué columnas tienen valores nulos.
- Usar describe() y anotar los valores promedio de xG.
- Renombrar las columnas con .str.upper().

A partir de la Lección 4

Se trabajará con el archivo FullShotsData.csv, que contiene los registros completos de disparos en fútbol.

```
# Cargar el archivo principal
df = pd.read_csv("FullShotsData.csv")
df.head()
```

Descripción de columnas clave:

Columna	Descripción
`id`	ID único del disparo
`minute`	Minuto del partido
`result`	Resultado del disparo (`Goal`, `Miss`, etc.)
`X`, `Y`	Coordenadas de disparo en el campo (0 a 1)
`xG`	Probabilidad de gol
`player`	Jugador que dispara
`h_a`	Local (`h`) o visitante (`a`)
`player_id`	Identificador del jugador
`situation`	Contexto: `OpenPlay`, `Corner`, etc.
`year`	Año del partido
`shotType`	Parte del cuerpo usada (`RightFoot`, `Header`, etc.)
`match_id`	ID del partido
`h_team`, `a_team`	Equipo local y visitante
`h_goals`, `a_goals`	Goles por equipo
`date`	Fecha del partido
`player_assisted`	Jugador que asistió (si aplica)
`lastAction`	Última acción previa al disparo (`Pass`, `Dribble`, etc.)

Lección 4: Selección y filtrado

```python
# Seleccionar una columna
df["player"]

# Seleccionar varias columnas
df[["player", "minute", "xG"]]

# Acceder a la primera fila
df.iloc[0]

# Acceder a filas por rango
df.iloc[5:10]

# Filtros simples
df[df["xG"] > 0.5]
df[df["result"] == "Goal"]
df[df["minute"] > 75]
```

```python
# Filtros múltiples con & y |
df[(df["xG"] > 0.5) & (df["result"] == "Goal")]
df[(df["minute"] < 30) | (df["result"] == "Saved")]

# Filtros por valores de lista
df[df["situation"].isin(["Corner", "FreeKick"])]

# Filtro por texto
df[df["player"].str.contains("Messi", na=False)]

# Filtro compuesto con .query()
df.query("xG > 0.4 and result == 'Goal'")
```

Ejercicios – Lección 4

- Filtrar todos los disparos con xG > 0.6 y result == "Saved".
- Mostrar player, minute, result y xG para los disparos entre los minutos 30 y 60.
- Mostrar disparos realizados por el equipo visitante (h_a == "a") luego del minuto 80.
- Mostrar los disparos con lastAction == "Dribble" y resultado "Goal".
- Filtrar disparos desde situaciones "Corner" o "FreeKick" con xG < 0.2.

Lección 5: Operaciones y estadísticas

```python
# Crear nueva columna con porcentaje
df["xG_pct"] = df["xG"] * 100

# Promedio, máximo, mínimo
df["xG"].mean()
df["minute"].max()

# Conteo de valores categóricos
df["result"].value_counts()
```

Ejercicios – Lección 5

- Crear la columna impacto = xG * minute.
- Mostrar el jugador con mayor xG.
- Contar cuántas veces aparece cada result.

Lección 6: Limpieza de datos

```
# Verificar valores nulos
df.isnull().sum()

# Rellenar nulos con texto
df["player_assisted"] = df["player_assisted"].fillna("Sin asistencia")

# Eliminar duplicados
df = df.drop_duplicates()
```

Ejercicios – Lección 6

- Rellenar nulos de columnas tipo texto con "Desconocido".
- Eliminar filas con nulos en xG.
- Eliminar duplicados y contar cuántos había.

Lección 7: Agrupaciones

```
# Agrupar por jugador y calcular promedio de xG
df.groupby("player")["xG"].mean()

# Número de disparos por resultado
df.groupby("result")["id"].count()

# Agrupar por shotType y calcular promedio
df.groupby("shotType")["xG"].mean()
```

Ejercicios – Lección 7

- Agrupar por año y mostrar promedio de xG.
- Mostrar el número total de disparos por jugador.
- Agrupar por situación (situation) y calcular media de xG.

Lección 8: Ordenar y combinar DataFrames

Ordenar registros con sort_values()

```
# Ordenar por una sola columna (xG en orden descendente)
df_ordenado = df.sort_values(by="xG", ascending=False)

# Mostrar los 5 disparos con mayor probabilidad de gol
df_ordenado.head(5)
```

```
# Ordenar por múltiples columnas: primero por año (ascendente), luego por jugador (alfabéticamente)
df_ordenado2 = df.sort_values(by=["year", "player"])
```

```
# Ordenar por minute de forma ascendente y por xG de forma descendente
df.sort_values(by=["minute", "xG"], ascending=[True, False])
```

Combinar DataFrames con merge()

merge() se utiliza para unir dos DataFrames usando una columna en común, como si se hiciera un JOIN en SQL.

```
# Supongamos que tenemos un DataFrame adicional con anotaciones
extra_info = pd.DataFrame({
    "id": [101, 102, 103],
    "comentario": ["Disparo clave", "Disparo débil", "Mal posicionado"]
})

# Unimos ambos DataFrames usando la columna "id"
df_merged = pd.merge(df, extra_info, on="id", how="left")
```

Explicación del parámetro how en merge()

how	Descripción
'inner'	Devuelve solo las filas que coinciden en ambos DataFrames.
'left'	Devuelve todas las filas del DataFrame izquierdo (df), y datos del derecho si coinciden.
'right'	Devuelve todas las filas del DataFrame derecho, y datos del izquierdo si coinciden.
'outer'	Devuelve todas las filas de ambos DataFrames. Si no hay coincidencia, rellena con NaN.

```python
# Merge tipo inner (solo los ids que estén en ambos DataFrames)
df_inner = pd.merge(df, extra_info, on="id", how="inner")

# Merge tipo outer (todos los registros de ambos DataFrames)
df_outer = pd.merge(df, extra_info, on="id", how="outer")
```

Ejercicios propuestos – Lección 8

- Ordenar el DataFrame por xG en orden descendente y mostrar los 5 primeros registros.
- Ordenar por minute (ascendente) y luego por xG (descendente).
- Crear un segundo DataFrame con las columnas id y nota, luego hacer un merge usando how="left".
- Aplicar un merge con how="inner" y mostrar cuántas filas coinciden.
- Realizar un merge con how="outer" y contar cuántos valores NaN hay en la columna combinada comentario.